UNIVERSITY INSTITUTE OF COMPUTING

Project Report

On

ShopFlow —
Inventory & Billing System

Program name: BCA

Subject name/Code: Java Programming Lab

(23CAP-352)

Submitted by:

Name: Ashish

UID: 23BCA10458

Section: 23BCA-8(B)

Submitted to:

Name: Mr. Divij Sumbly

Assistant Professor

Contents

1 Introduction **1**

1.1 Problem Statement 1

2 Aim of the Project **2**

3 Objectives **3**

4 Technology Used 5

4.1 Programming Language — Java SE 5

4.2 Libraries and APIs Used 5

4.3 Development Tools 6

4.4 No External Dependencies 6

5 System Requirements **7**

5.1 Hardware Requirements 7

5.2 Software Requirements 7

6 Algorithm / Flow **8**

6.1 Overall Application Flow 8

6.2 Product Management Algorithm 8

6.3 Billing Algorithm 9

6.4 PDF Generation Algorithm 9

6.5 Low Stock Detection Logic 10

7 Code — Key Snippets **11**

7.1 Data Models 11

7.2 Saving and Loading Data 11

7.3 Billing — Add to Cart with Stock Validation 12

7.4 Auto-Save on Application Exit 13

7.5 Reusable UI Helper Methods 13

8 Screenshots / Output **14**

8.1 Dashboard 14

8.2 Products Panel 14

8.3 Billing Panel 15

8.4 Generated PDF Invoice 15

9 Learning Outcomes **16**

10 GitHub & Useful Links **18**

10.1 Project Repository on GitHub 18

10.2 How to Clone and Run from GitHub 18

10.3 Reference Links 19

11 Conclusion **20**

11.1 Future Scope 20

Chapter 1

Introduction

ShopFlow is a desktop-based Inventory and Billing Management System developed using **Java** and the **Swing** GUI toolkit. It is a single-file application designed to help small and medium-sized retail shops manage their daily operations efficiently — without needing an internet connection or any external database.

The application allows a shopkeeper to manage products, generate GST-included customer bills as PDF invoices, and track complete sales history — all from a simple, easy-to-use desktop interface.

1.1 Problem Statement

Small shopkeepers often face these common problems in daily business:

- Tracking product stock manually leads to errors and unexpected stock-outs.
- Writing bills by hand is slow, error-prone, and unprofessional.
- No record of past sales makes it hard to track revenue, profit, or customer history.
- There is no easy way to know which items are running low in stock before it is too late.
- Existing software is either too expensive, too complicated, or requires internet access.

ShopFlow was built to solve all of the above problems with a **simple, free, offline desktop application** that any shopkeeper can use without technical knowledge.

Chapter 2

Aim of the Project

The main aim of **ShopFlow** is to develop a complete, lightweight, offline desktop application that gives a shop owner full control over inventory and billing from one place.

The application aims to:

- **Manage Products** — Add, edit, delete, and search products with full details including price, GST rate, stock, and minimum quantity.
- **Generate PDF Invoices** — Create professional customer bills automatically with itemized GST calculations and save them as PDF files.
- **Track Sales History** — Maintain a complete record of all transactions in-cluding date, customer, items sold, revenue, and profit.
- **Show Low Stock Alerts** — Automatically highlight products that have fallen below the minimum stock level.
- **Display a Dashboard** — Provide an instant summary of key business num-bers: total products, inventory value, revenue, and profit.
- **Auto-Save All Data** — Save and restore all data automatically so the shop-keeper never loses any information.
- **Work Completely Offline** — Operate with zero dependency on the internet, cloud, or any external database.

Chapter 3

Objectives

The specific objectives of this project are listed below:

1. **Product Management**

   Allow users to add new products with all required details: name, category, barcode, buy price, sell price, GST rate, current stock quantity, and minimum stock level. Support editing and deletion of any product.

2. **Category Management**

   Provide default built-in categories (Electronics, Groceries, Clothing, etc.) and allow users to add their own custom categories at any time.

3. **Billing System**

   Build a cart-based billing interface where products can be searched, added to a cart with any quantity, and a complete PDF invoice can be generated for the customer.

4. **GST Calculation**

   Automatically compute GST for every item based on its configured GST percentage rate. Show a clear breakdown of subtotal, total GST, and grand total on every bill.

5. **PDF Invoice Generation**

   Generate formatted, print-ready PDF invoices and save them to an invoices/ folder – without using any third-party PDF library.

6. **Sales History**

   Maintain a complete list of all past transactions. Allow the user to re-open any previous invoice as a PDF at any time.

7. Dashboard and Analytics

Show key business metrics on the dashboard: total number of products, total inventory value, number of low-stock items, total revenue, and total profit.

8. Data Persistence

Automatically save and restore all data (products, sales, categories, counters) using Java Serialization so data is never lost between sessions.

9. Offline and Standalone Operation

The complete application must run without internet, without a database server, and without any installation beyond the Java Runtime Environment.

Chapter 4

Technology Used

4.1 Programming Language — Java SE

This project is developed entirely in **Java Standard Edition (Java SE 8 or above)**. Java is a class-based, object-oriented programming language that is platform-independent, meaning the same compiled code runs on Windows, Linux, and macOS without any changes.

4.2 Libraries and APIs Used

All libraries used in this project are part of the **Java Standard Library** — no external JAR files or third-party dependencies are needed.

Library / API	Package	Purpose
Java Swing	javax.swing.*	Building the full desktop GUI — windows, buttons, tables, dialogs
Java AWT	java.awt.*	Layout managers, fonts, and mouse/keyboard events
Java IO / NIO	java.io.*, java.nio.*	Reading and writing files, generating the PDF
Java Serialization	java.io.Serializable	Saving and loading all data to a .ser file
Java Time	java.time.*	Generating timestamps and formatted dates for bills
Java Streams	java.util.stream.*	Filtering, searching products, and computing totals
Java Collections	java.util.*	Managing lists of products, sales, and categories

4.3 Development Tools

- **JDK 8 or above** — Java Development Kit for compiling the source file
- **VS Code / IntelliJ IDEA** — Code editor used for writing and testing the program
- **Command Prompt / Terminal** — Used to compile and run the application
- **Git and GitHub** — For version control and storing the source code online

4.4 No External Dependencies

This project uses **zero external JAR files or libraries**. Even the PDF generation is done by writing raw PDF file syntax using Java's built-in `ByteArrayOutputStream`. This makes the project extremely simple to compile and run on any machine that has Java installed.

Chapter 5

System Requirements

5.1 Hardware Requirements

Component	Minimum Requirement	
Processor	Intel Core i3 or equivalent (1 GHz or faster)	
RAM	512 MB (1 GB recommended)	
Storage	100 MB free disk space	
Display	1024 x 768 resolution or higher	
Input Devices	Keyboard and Mouse	

5.2 Software Requirements

Software	Details	
Operating System	Windows 7 / 8 / 10 / 11, Linux, or macOS	
Java Runtime (JRE)	Java 8 or above	
Java Development Kit	JDK 8 or above (required for compilation)	
PDF Viewer	Any PDF reader (Adobe Acrobat, browser, etc.)	
Terminal / CMD	To compile and run the .java file	

Chapter 6

Algorithm / Flow

6.1 Overall Application Flow

1. **Start:** The application launches and checks if a saved data file shopflow_data.ser exists.

2. **Load Data:** If the file exists, all products, sales, and categories are loaded from it automatically.

3. **Show Main Window:** The main window opens with the Dashboard panel displayed by default.

4. **User Navigates:** User clicks any of the 4 sidebar buttons to switch between Dashboard, Products, Billing, and Sales History panels.

5. **Perform Operations:** User adds/edits products, generates bills, and views sales history as needed.

6. **Auto-Save on Exit:** When the user closes the application, all data is saved back to shopflow_data.ser automatically via a shutdown hook.

6.2 Product Management Algorithm

1. User clicks **"+ Add Product"** button.

2. A form dialog opens with input fields: Name, Category, Barcode, Buy Price, Sell Price, GST%, Stock Qty, Min Stock.

3. System validates all inputs — name must not be empty, all numeric values must be valid and non-negative.

4. If validation passes, a new Product object is created with a unique auto-incremented ID and added to the product list.

5. The product table refreshes to show the new entry.

6. For Edit: the same form is opened with fields pre-filled with the existing product's values.

7. For Delete: user is shown a confirmation dialog before the product is removed.

6.3 Billing Algorithm

1. User switches to the **Billing** panel. The left side shows all in-stock products.

2. User searches for a product, selects it, enters the desired quantity, and clicks
   "Add to Cart".

3. System checks: existingCartQty + newQty must not exceed the available stock. If it does, an alert is shown.

4. If the product is already in the cart, the quantity is increased; otherwise a new cart entry is created.

5. The cart table updates showing each item's unit price, quantity, GST amount, and subtotal.

6. The running totals (Subtotal, Total GST, Grand Total) update live at the bottom.

7. User types the customer name and clicks **"Generate PDF Invoice"**.

8. System deducts the sold quantities from each product's stock, creates a Sale
   record, and calls the PDF generator.

9. Cart is cleared and all panels refresh.

6.4 PDF Generation Algorithm

1. Build a PDF content stream as a text string using standard PDF operators (BT, ET, Td, Tf, Tj).

2. Format and write each bill line: Product Name, Unit Price, Quantity, GST amount, and Subtotal.

3. Append the totals section: Subtotal, Total GST, and Grand Total.

4. Construct 6 required PDF objects: Catalog, Pages, Page, Content stream, and two Font objects.

5. Build the cross-reference table (xref) recording the byte offset of each object.

6. Write the PDF trailer and the %%EOF end marker.

7. Save the complete byte array to invoices/<filename>.pdf and open it via the system's default PDF viewer.

6.5 Low Stock Detection Logic

```
// Inside the Product class
boolean low() {
    return qty <= min;
    // returns true when stock is at or below minimum threshold
}
// In Dashboard - count all low stock products
long lowCount = products.stream()
                        .filter(Product::low)
                        .count();
```

Listing 6.1: Low stock check in the Product class

Chapter 7

Code — Key Snippets

7.1 Data Models

```
// Product - stores all details of one inventory item
static class Product implements Serializable {
    int id;
    String name, cat, barcode;
    double buyPrice, sellPrice, gstRate;
    int qty, min;

    boolean low() { return qty <= min; }          // is stock low?
    double val() { return buyPrice * qty; }       // total
        inventory value
}

// BillItem - represents one line item in a customer bill
static class BillItem implements Serializable {
    Product p;
    int qty;

    double sub()     { return p.sellPrice * qty; }
    double gst()     { return sub() * (p.gstRate / 100.0); }
    double profit() { return (p.sellPrice - p.buyPrice) * qty; }
}
```

Listing 7.1: Product and BillItem data model classes

7.2 Saving and Loading Data

```
// SAVE - called automatically when the application closes
static void saveData() {
    try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("shopflow_data.ser"))) {
        oos.writeObject(products);              // save all products
        oos.writeObject(sales);                 // save all sales
            records
        oos.writeInt(PID);                      // save next product ID
            counter
        oos.writeInt(BILL);                     // save next bill
            number counter
        oos.writeObject(customCategories);      // save custom
            categories
    } catch (Exception e) {
```

```
            System.err.println("Failed to save: " + e.getMessage());
        }
    }

    // LOAD - called when the application starts
    static void loadData() {
        File f = new File("shopflow_data.ser");
        if (f.exists()) {
            try (ObjectInputStream ois = new ObjectInputStream(
                        new FileInputStream(f))) {
                products        = (List<Product>) ois.readObject();
                sales           = (List<Sale>)    ois.readObject();
                PID             = ois.readInt();
                BILL            = ois.readInt();
                customCategories = (List<String>) ois.readObject();
            } catch (Exception e) {
                System.err.println("Failed to load: " +
                    e.getMessage());
            }
        }
    }
```

Listing 7.2: Data persistence using Java Serialization

7.3 Billing — Add to Cart with Stock Validation

```
addBtn.addActionListener(e -> {
    int qty = Integer.parseInt(qtyField.getText().trim());
    int id  = Integer.parseInt(selected.split("\\|")[0].trim());

    products.stream().filter(x -> x.id ==
        id).findFirst().ifPresent(pr -> {

        // calculate how many units are already in the cart
        int inCart = cart.stream()
                        .filter(ci -> ci.p.id == pr.id)
                        .mapToInt(ci -> ci.qty)
                        .sum();

        // check if requested qty exceeds available stock
        if (inCart + qty > pr.qty) {
            alert(" Not enough stock! Available : " + (pr.qty -
                inCart));
            return;
        }

        // if already in cart increase qty, else add new BillItem
```

```
        boolean found = false;
        for (BillItem bi : cart)
            if (bi.p.id == pr.id) { bi.qty += qty; found = true;
                break; }
        if (!found)
            cart.add(new BillItem(pr, qty));

        updateCart();
    });
});
```

Listing 7.3: Cart validation and add-to-cart logic

7.4 Auto-Save on Application Exit

```
public static void main(String[] args) {
    loadData();    // load previously saved data when app starts

    // this block runs automatically when the user closes the
        window
    Runtime.getRuntime().addShutdownHook(new Thread(() ->
        saveData()));

    // launch the GUI on the Event Dispatch Thread
    SwingUtilities.invokeLater(() -> new
        MainWin().setVisible(true));
}
```

Listing 7.4: Shutdown hook ensures data is always saved on exit

7.5 Reusable UI Helper Methods

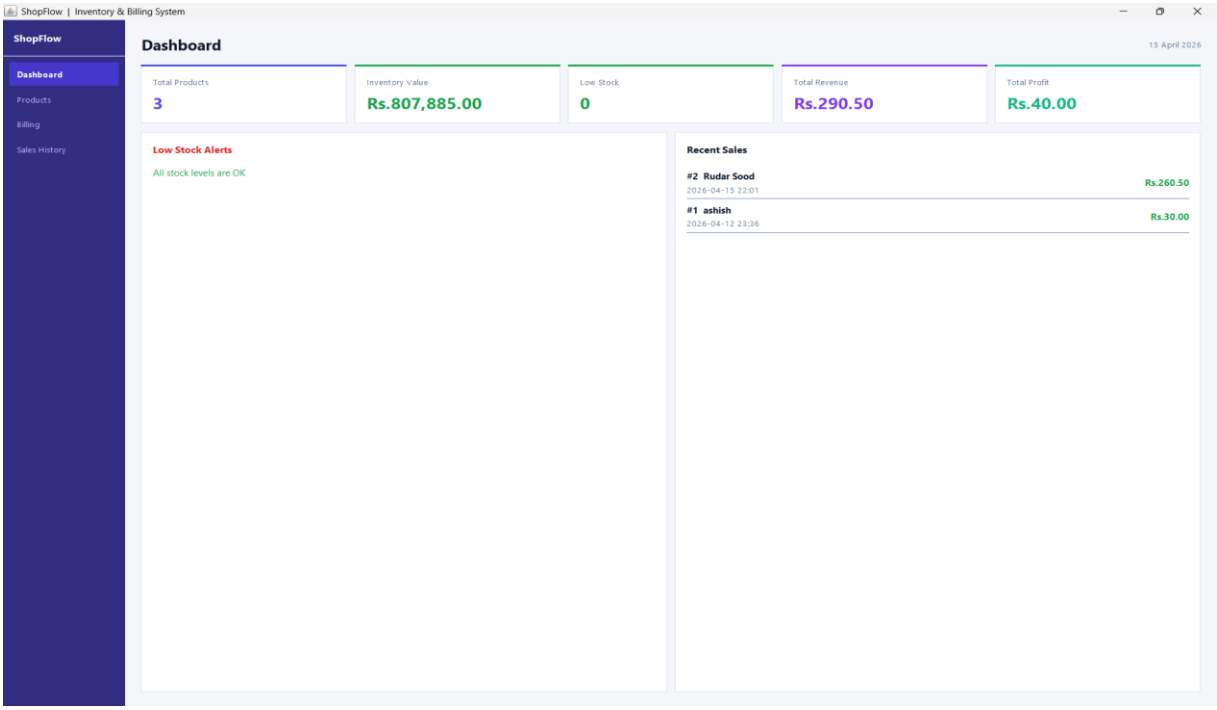
```
// Creates a styled button with a hover effect
static JButton btn(String text, Color bg) {
    JButton b = new JButton(text);
    b.setFont(FBD);
    b.setBackground(bg);
    b.setForeground(Color.WHITE);
    b.setFocusPainted(false);
    b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    Color darker = bg.darker();
    b.addMouseListener(new MouseAdapter() {
        public void mouseEntered(MouseEvent e) {
            b.setBackground(darker); }
        public void mouseExited(MouseEvent e) {
            b.setBackground(bg);     }
    });
```

Listing 7.5: Shared utility methods used across all panels

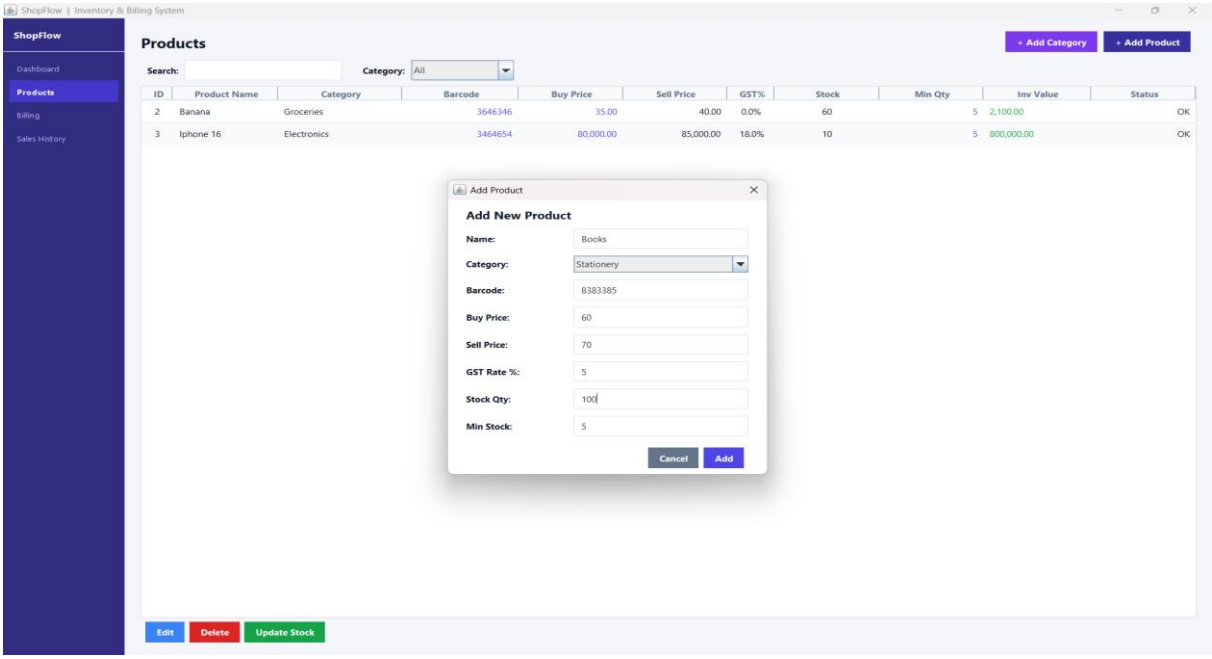
Chapter 8

Screenshots / Output

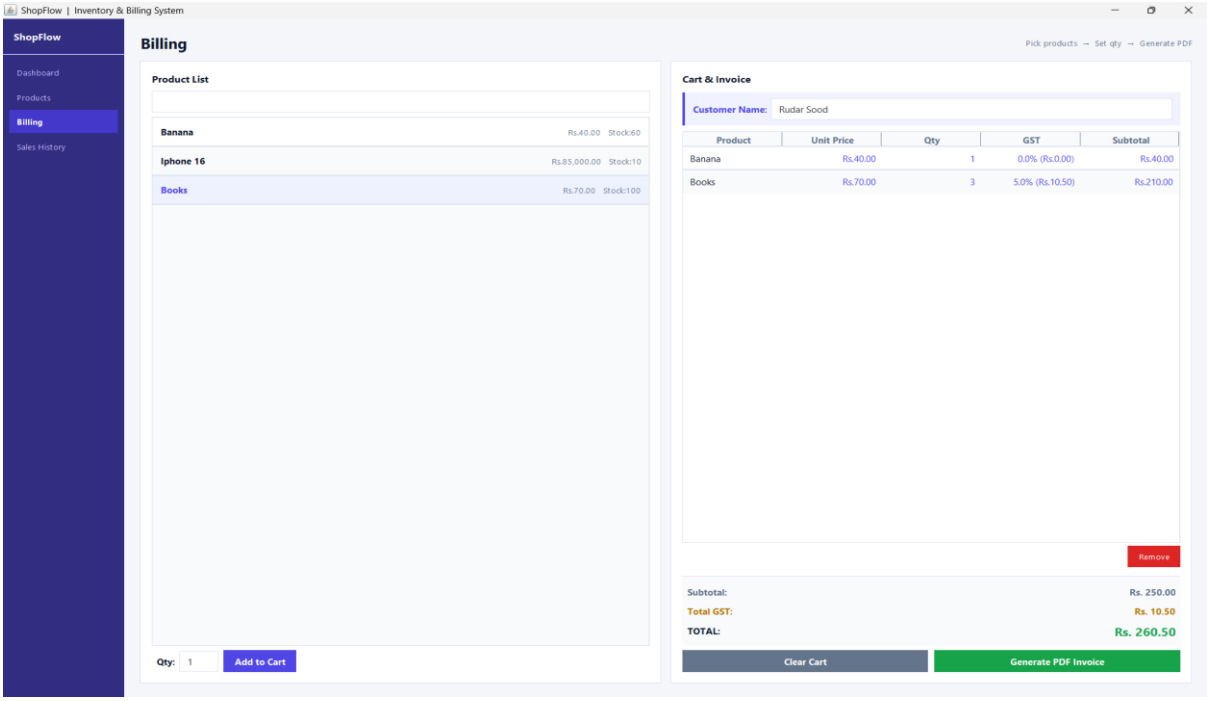
8.1 Dashboard



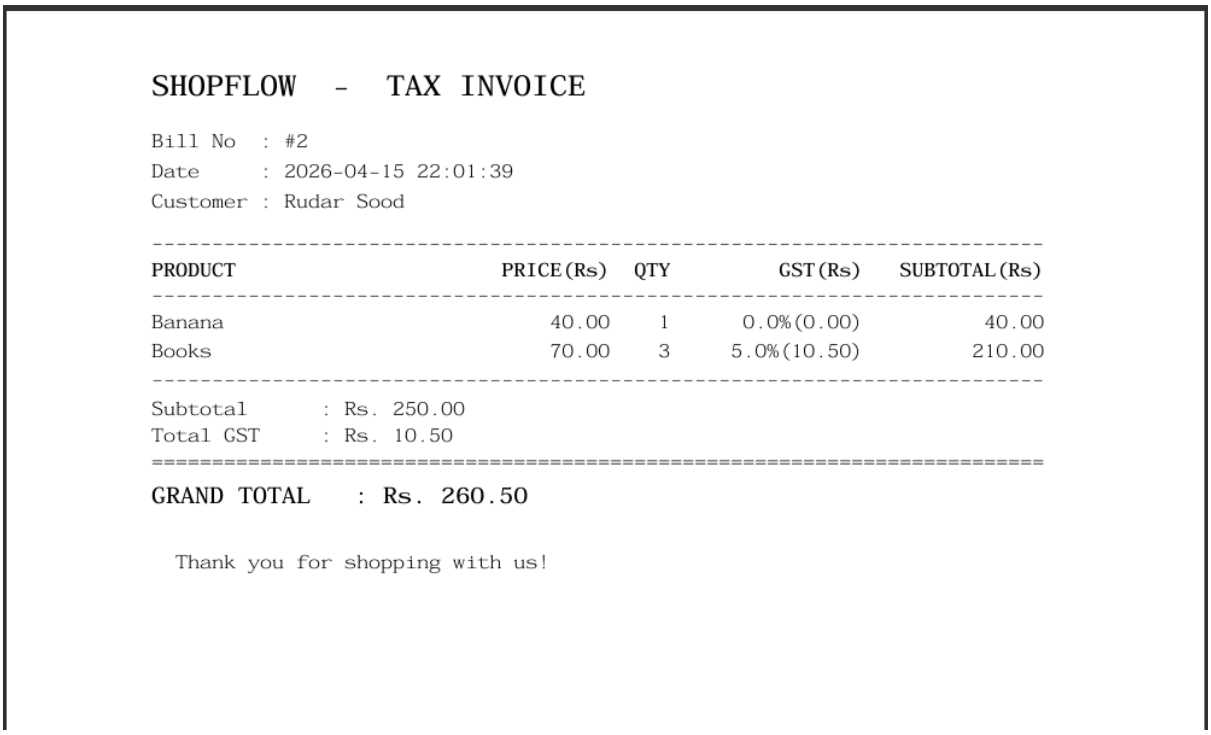
8.2 Products Panel



8.3 Billing Panel



8.4 Generated PDF Invoice



SHOPFLOW - TAX INVOICE

Bill No : #2
Date : 2026-04-15 22:01:39
Customer : Rudar Sood

PRODUCT	PRICE(Rs)	QTY	GST(Rs)	SUBTOTAL(Rs)
Banana	40.00	1	0.0%(0.00)	40.00
Books	70.00	3	5.0%(10.50)	210.00

Subtotal : Rs. 250.00
Total GST : Rs. 10.50

GRAND TOTAL : Rs. 260.50

Thank you for shopping with us!

Chapter 9

Learning Outcomes

By completing this project, the following practical skills and concepts were learned and applied:

1. **Java Swing GUI Development**

   Learned to build complete desktop user interfaces using JFrame, JPanel, JTable,
   JDialog, JList, CardLayout, and many other Swing components.

2. **Event-Driven Programming**

   Understood how GUI applications respond to user actions such as button clicks, text input, and table row selection using ActionListener, MouseListener, and DocumentListener.

3. **Data Persistence with Java Serialization**

   Implemented file-based data saving and loading using ObjectOutputStream and ObjectInputStream, allowing all data to survive application restarts without a database.

4. **Object-Oriented Design**

   Designed clean, reusable data model classes (Product, BillItem, Sale) and organised related logic using static inner classes.

5. **Java Streams and Lambda Expressions**

   Used the Stream API to filter product lists, count low-stock items, and sum totals in a concise and readable way.

6. **PDF File Generation from Scratch**

   Learned the internal structure of PDF files (objects, cross-reference tables, content streams) and generated them programmatically using only Java's built-in IO classes.

7. Functional Interfaces

Created a custom `@FunctionalInterface` called `SimpleDocListener` to use `DocumentListener` as a lambda expression, reducing boilerplate code.

8. GST and Business Logic Implementation

Implemented real-world tax calculation logic including per-item GST rates, profit tracking per sale, and cumulative revenue reporting.

9. Software Architecture in a Single File

Learned how to structure a large application in one file using shared utility methods, global data stores, and multiple inner panel classes.

10. Real-World Problem Solving

Translated a real business requirement (retail shop billing and inventory) into fully working software, handling practical edge cases like stock validation, negative value prevention, and data integrity on exit.

Chapter 10

GitHub & Useful Links

10.1 Project Repository on GitHub

The complete source code for this project is hosted on GitHub and can be accessed at:

GitHub Repository:

https://github.com/NotAshish7/ShopFlow-Inventory-Billing-System.git

10.2 How to Clone and Run from GitHub

```
# Clone the repository to your local machine
git clone
    https://github.com/yourusername/shopflow-inventory-system.git

# Navigate into the project folder
cd shopflow-inventory-system

# Create output directory and compile
mkdir bin
javac -d bin -encoding UTF-8 InventoryManagementSystem.java

# Run the application
java -cp bin InventoryManagementSystem
```

Listing 10.1: Clone and run the project from GitHub

10.3 Reference Links

Resource	Link	
Java Official Documentation	https://docs.oracle.com/en/java/	
Java Swing Tutorial	https://docs.oracle.com/javase/tutorial/uiswing/	
Java Serialization Guide	https://docs.oracle.com/javase/8/docs/technotes/guides/serialization/	
Java Stream API	https://docs.oracle.com/javase/8/docs/api/java/util/stream/	
PDF File Format Reference	https://www.adobe.com/devnet/pdf/pdf_reference.html	
GitHub	https://github.com	

Chapter 11

Conclusion

ShopFlow is a fully functional, offline desktop application that successfully solves the real-world problem of inventory and billing management for small retail shops. The entire project is built in **Java** using only the standard library — no external frameworks, no internet connection, and no database server are required.

The system meets all its objectives:

- Products can be added, edited, deleted, and searched easily through a clean interface.
- Customer bills are generated with automatic GST calculations and saved as proper PDF invoices.
- A complete sales history is maintained with revenue and profit tracked for every transaction.
- Low stock alerts ensure the shop never unexpectedly runs out of important items.
- All data is saved automatically when the application closes and restored on the next launch.

This project demonstrates that a complete, professional-quality software application can be built using just the core Java platform. It also strengthened practical understanding of GUI programming, file handling, object-oriented design, and real-world business logic.

11.1 Future Scope

The following features can be added in future versions:

1. **Login System** — Add admin and cashier roles with password protection for security.
2. **Database Support** — Replace file serialization with SQLite or MySQL for better data management and multi-user support.

3. **Barcode Scanner Integration** — Support physical barcode scanners to speed up the billing process.

4. **Sales Reports** — Generate monthly and weekly PDF reports showing best-selling products and profit trends.

5. **Supplier Management** — Track purchase orders and supplier contact de-tails.

6. **Multi-language Support** — Add support for regional languages such as Hindi or Punjabi.

7. **Cloud Backup** — Automatically back up all data to Google Drive or Drop-box.

8. **Mobile Application** — Develop an Android companion app for checking inventory on the go